CSC 4814 & 6814 Efficient AI

Lecture 05

Dr. Haoxin Wang

Assistant Professor, GSU

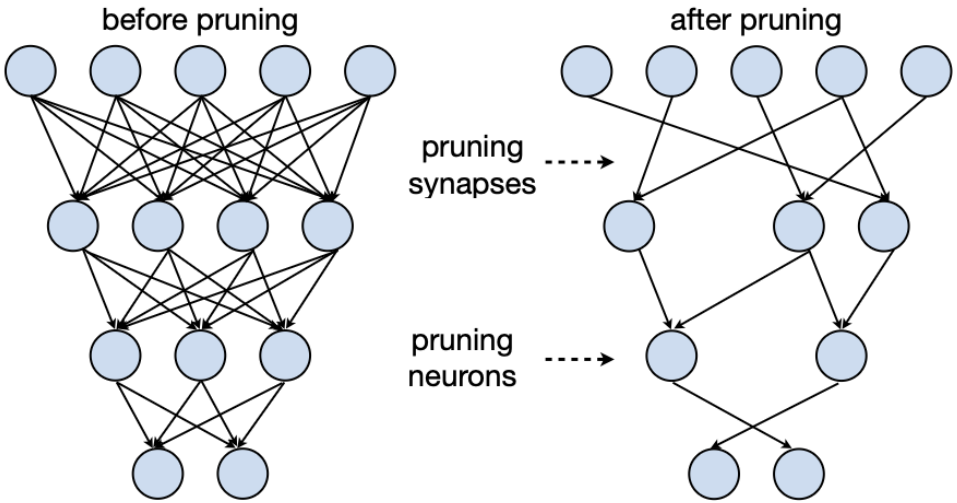
https://www.amai-gsu.us/

haoxinwang@gsu.edu

This course draws inspiration and selected materials from Prof. Song Han's MIT course on TinyML and Efficient Deep Learning Computing. Credit is gratefully acknowledged.

Agenda for Lecture 5

- Pruning and Sparsity



Neural Network Pruning

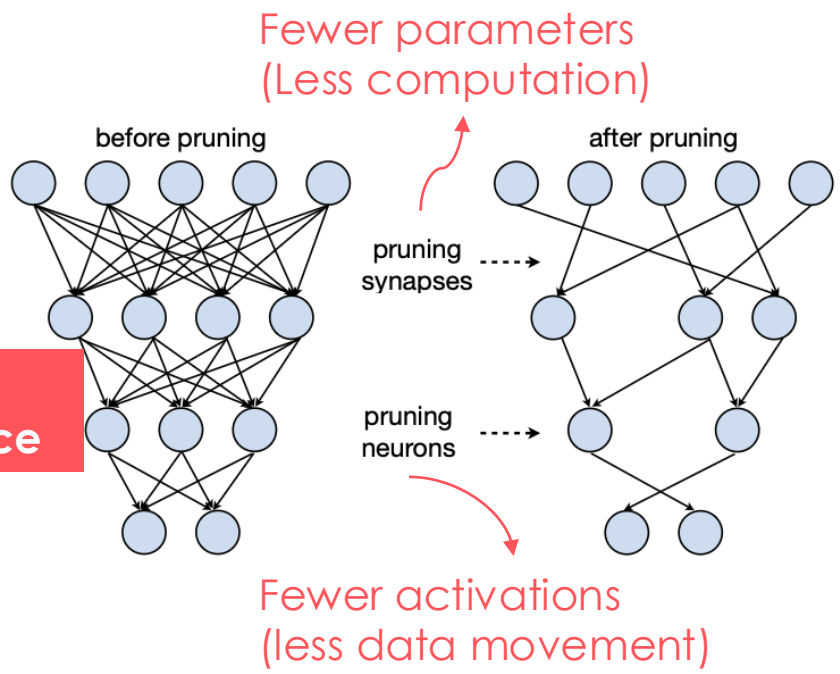
- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?

Cutting away weights or activations that contribute very little to the model's performance

- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?

Question: how do we measure the influence of pruning on the model's performance?

models?

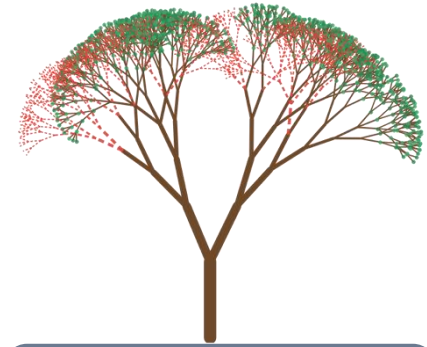


Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Pruning: Trim What You Don't Need

Remove the least useful weights while keeping the important structure intact

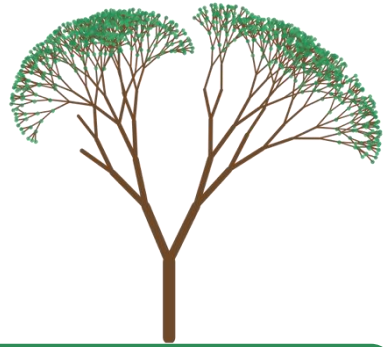
Think of pruning like trimming a tree. We cut away branches that aren't pulling their weight, and the tree keeps its shape, stays healthy, and grows back stronger.



(Before) red = low-value branches

Prune

cut ~40% of branches



(After) trimmed, structure intact

In a NN: branches = weights. Pruning removes the least useful weights → a smaller, faster model with little accuracy loss.

Neural Network Pruning

- In general, we could formulate the pruning as follows:

$$\arg\min_{\mathbf{W}_P} L(\mathbf{x}; \mathbf{W}_P)$$

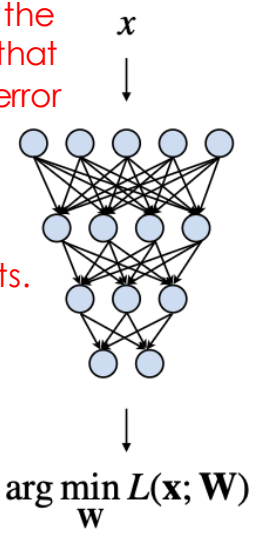
subject to

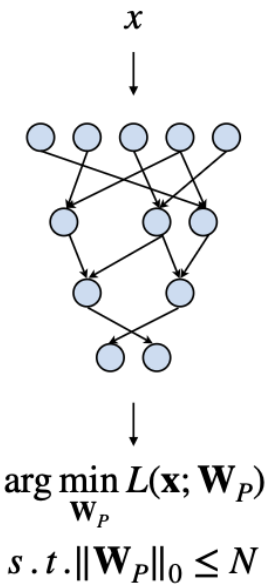
$$\|\mathbf{W}_p\|_0 < N$$

Among all possible values of the weights, find the weights W_p that make the model prediction error as small as possible.

You are only allowed to keep at most N nonzero weights.

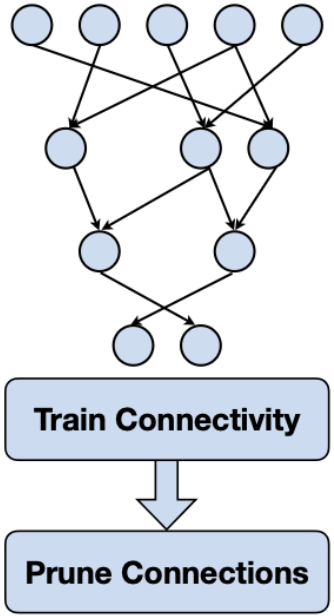
- L represents the objective function for neural network training;
- $\mathbf{x}$ is input, $\mathbf{W}$ is original weights, $\mathbf{W}_P$ is pruned weights;
- $\|\mathbf{W}_p\|_0$ calculates the #nonzeros in W_P, and N is the target #nonzeros.



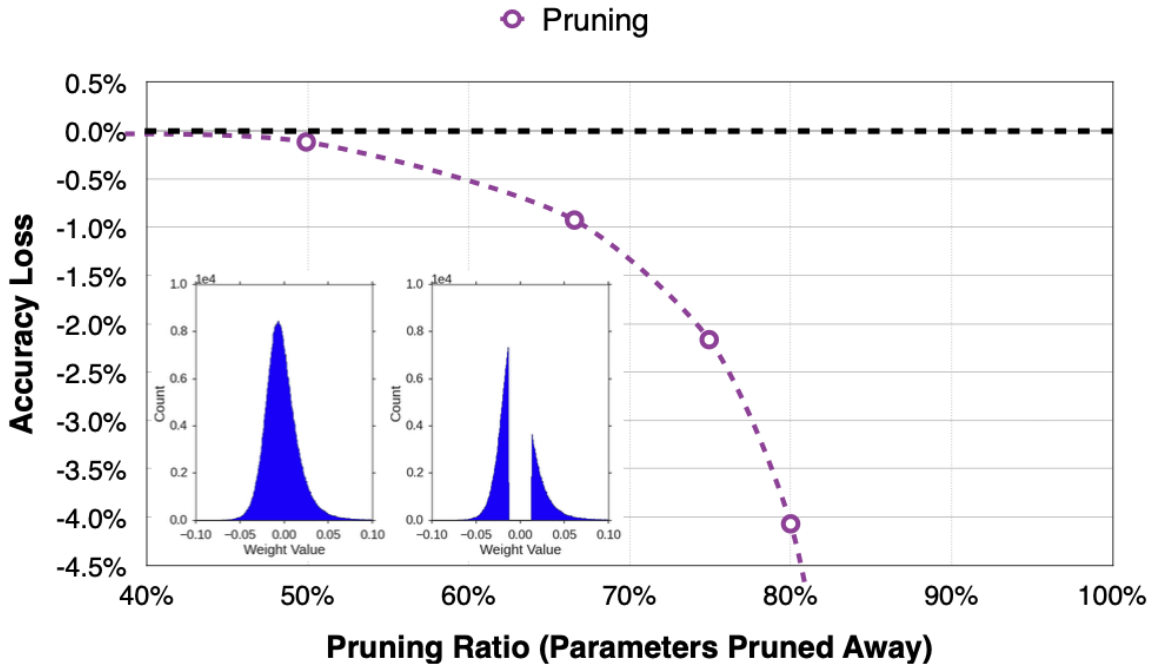


Neural Network Pruning

Make neural network smaller by removing parameters and activations



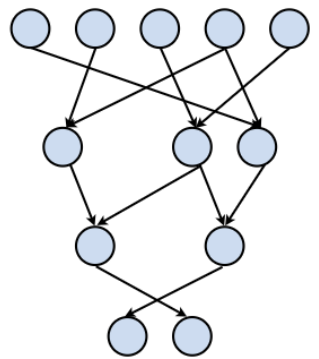
AlexNet



The more we prune, the less accurate.

Neural Network Pruning

Make neural network smaller by removing parameters and activations



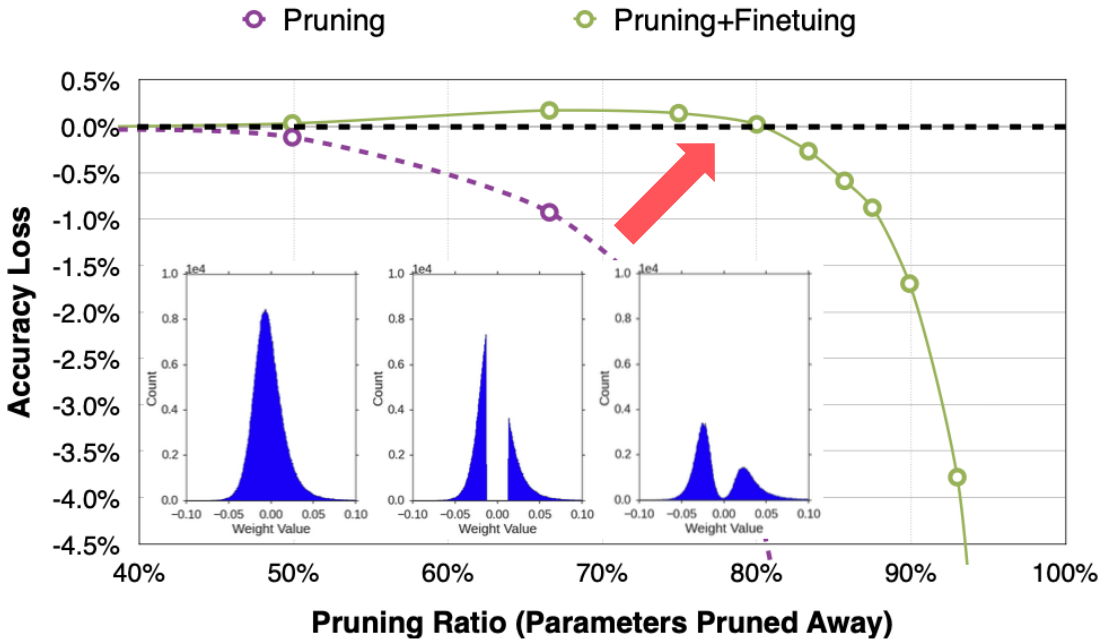
Train Connectivity

Prune Connections

Train Weights

Learning

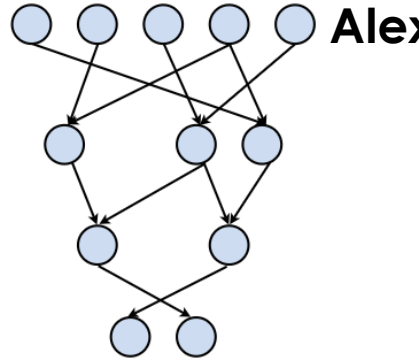
AlexNet



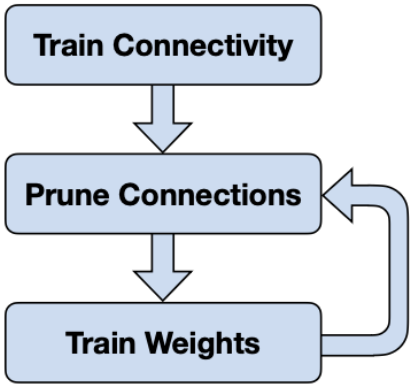
We can actually re-train the remaining weights to recover that accuracy.

Neural Network Pruning

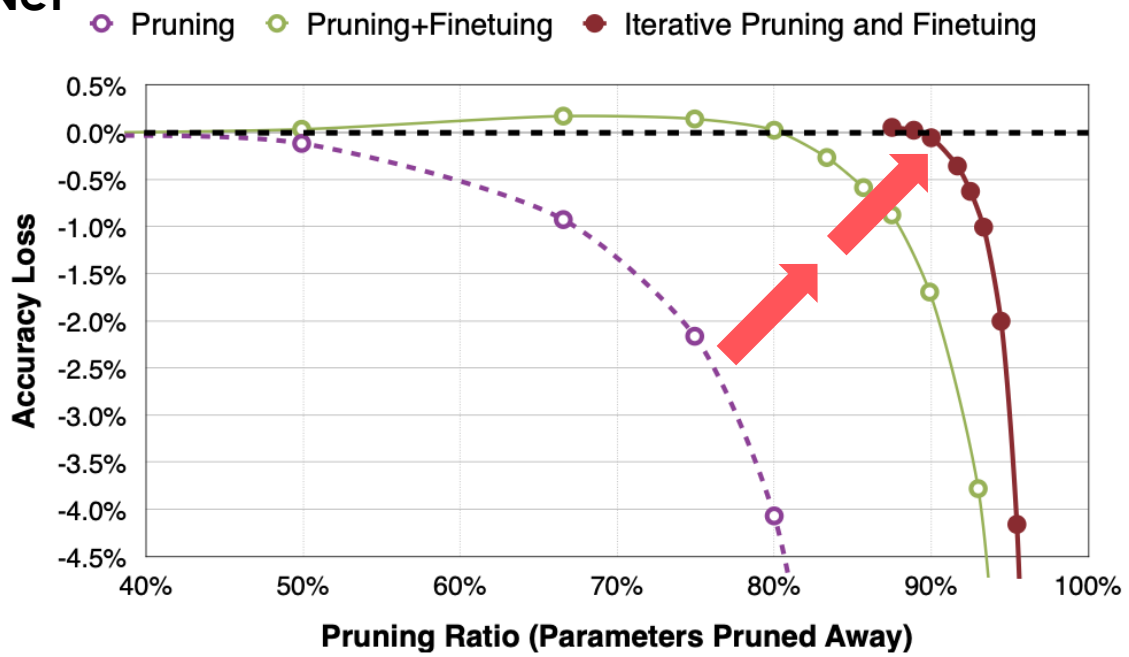
Make neural network smaller by removing parameters and activations



AlexNet



Learning Both W



Do this process iteratively: pruning → retraining → pruning → retraining → …

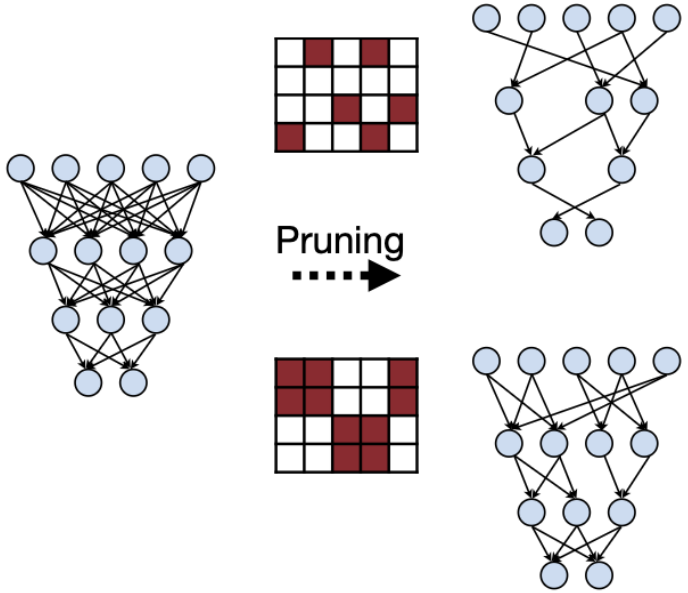
Pruning Granularity

Pruning can be performed at different granularities, from structured to non-structured.



Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?



Pruning granularity means: what is the "unit" that we remove from the neural network?

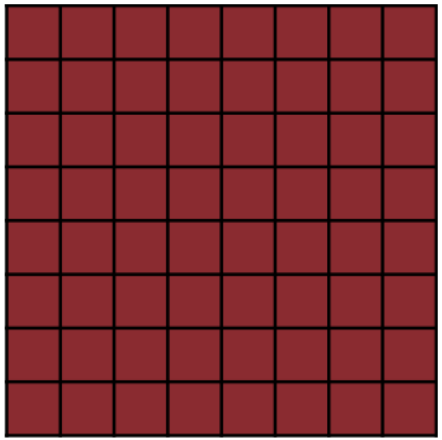
- **individual weights?**
- **a group of weights?**
- **a channel/filter?**

- How should we improve performance of pruned models?

Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Pruning at Different Granularities

A simple example of 2D weight matrix

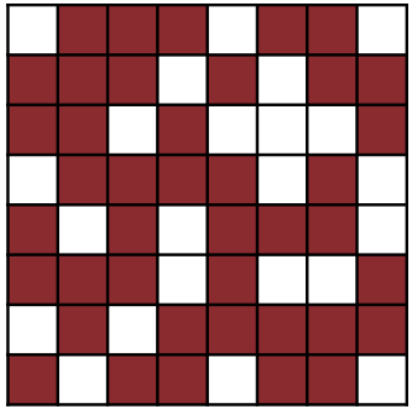


Dense 2D weight matrix

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

Pruning at Different Granularities

A simple example of 2D weight matrix



Fine-grained/Unstructured

- More flexible pruning index choice
- Hard to accelerate (irregular)

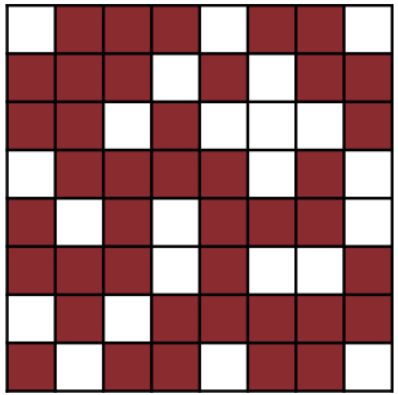
$$\begin{bmatrix} w_{11} & 0 & w_{13} \\ 0 & w_{22} & 0 \\ w_{31} & 0 & w_{33} \end{bmatrix}$$

Preserved

Pruned

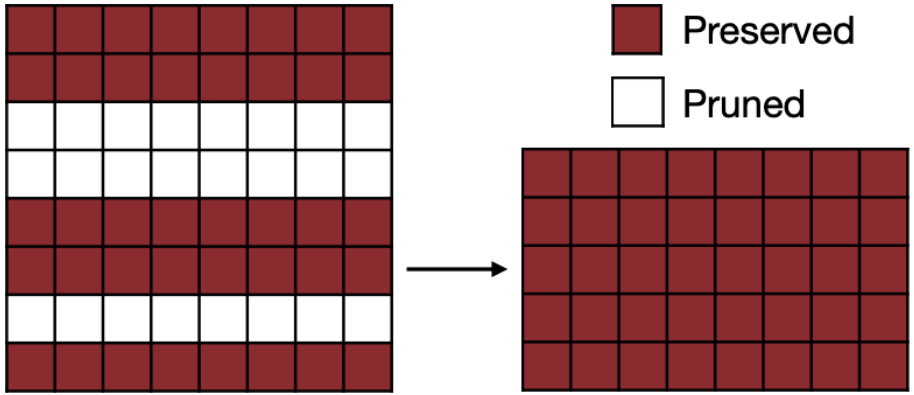
Pruning at Different Granularities

A simple example of 2D weight matrix



Fine-grained/Unstructured

- More flexible pruning index choice
- Hard to accelerate (irregular)



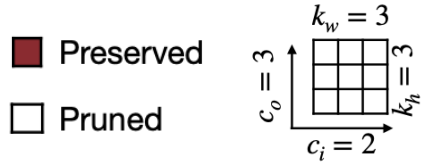
Coarse-grained/Structured

- Less flexible pruning index choice (a subset of the fine-grained case)
- Easy to accelerate (just a smaller matrix!)

Pruning at Different Granularities

The case of convolutional layers

- Some of the commonly used pruning granularities

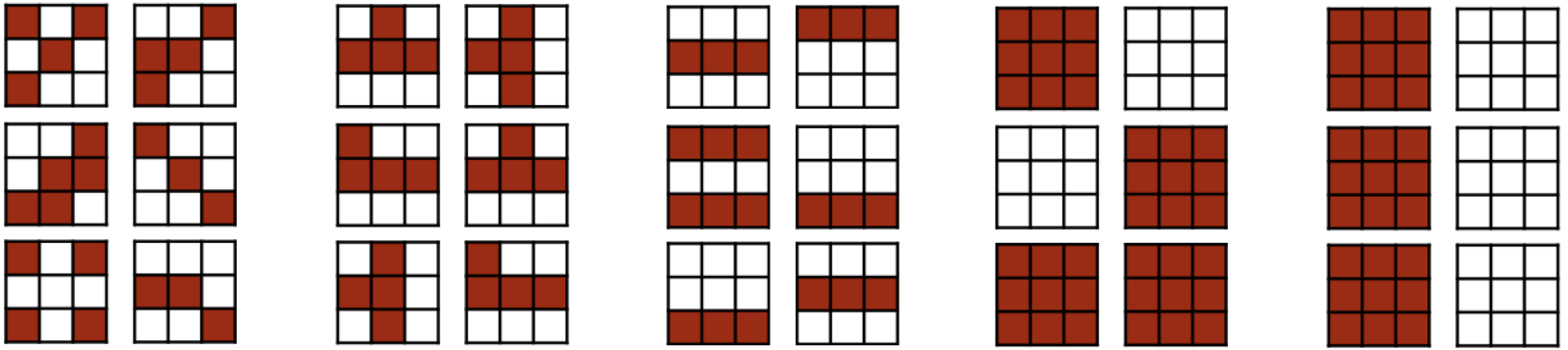


Pros?

Cons?

Irregular

Regular



Fine-grained Pruning

Pattern-based Pruning

Vector-level Pruning

Kernel-level Pruning

Channel-level Pruning

like Tetris :)

Exploring the granularity of sparsity in convolutional neural networks [Mao *et al.*, CVPR-W]

Pruning at Different Granularities

Let's look into some cases

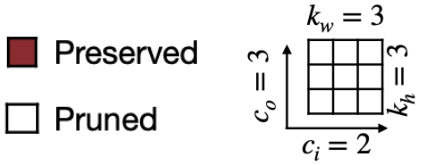
- **Fine-grained Pruning** (the case we show before)
  - Flexible pruning indices
  - Usually larger compression ratio since we can flexibly find "redundant" weights (we will later discuss how we find them)

Neural Network	#Parameters		
	Before Pruning	After Pruning	Reduction
AlexNet	61 M	6.7 M	**9** ×
VGG-16	138 M	10.3 M	**12** ×
GoogleNet	7 M	2.0 M	**3.5** ×
ResNet50	26 M	7.47 M	**3.4** ×

Pruning at Different Granularities

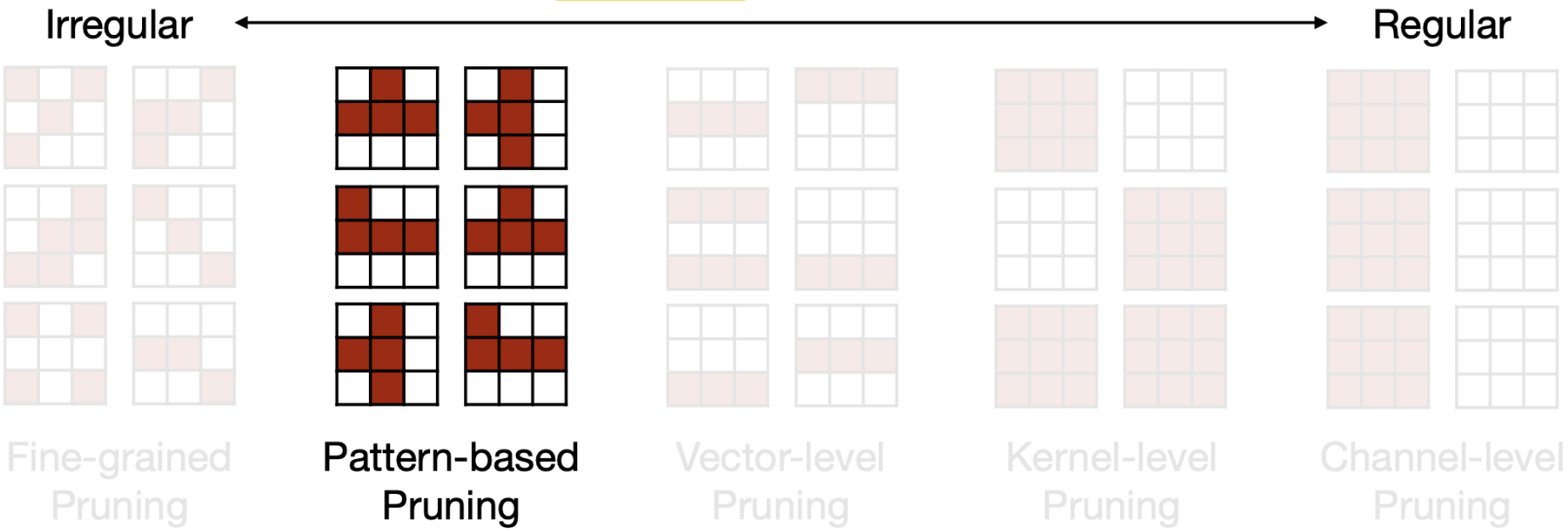
The case of convolutional layers

- Some of the commonly used pruning granularities



Pros?

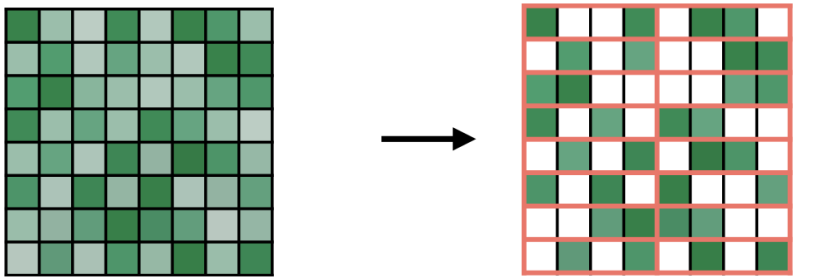
Cons?



Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**

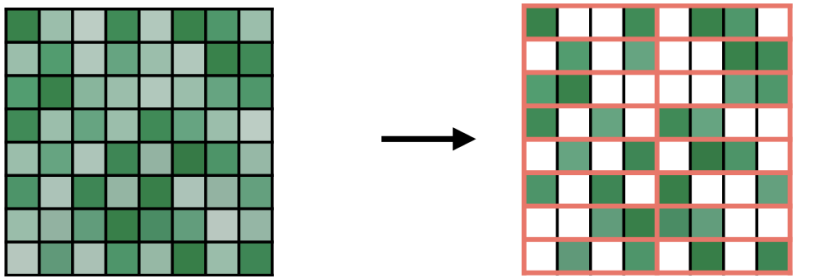


Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT

Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**
  - N:M sparsity means that in each contiguous M elements, N of them is pruned



Dense Matrix

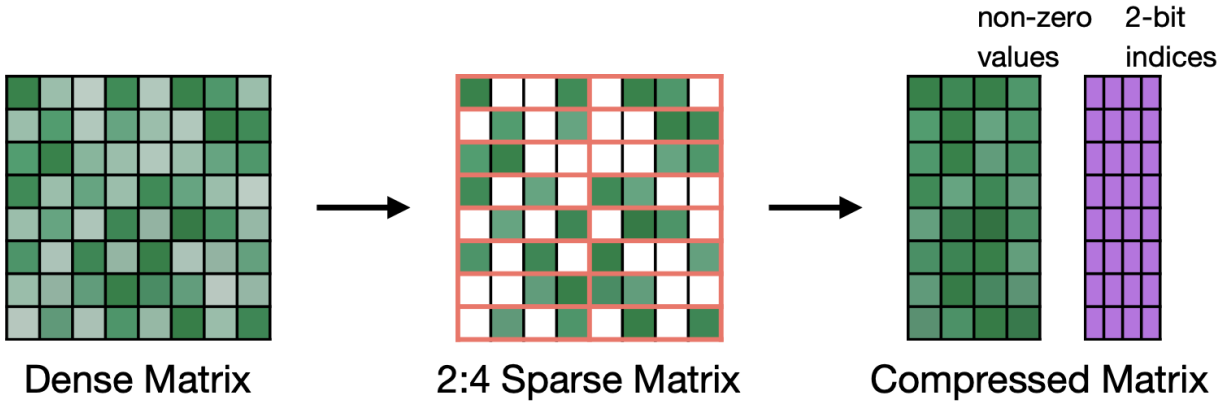
2:4 Sparse Matrix

Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT

Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**
  - N:M sparsity means that in each contiguous M elements, N of them is pruned
  - A classic case is 2:4 sparsity (50% sparsity)
  - It is supported by NVIDIA's Ampere GPU Architecture, which delivers up to 2x speed up



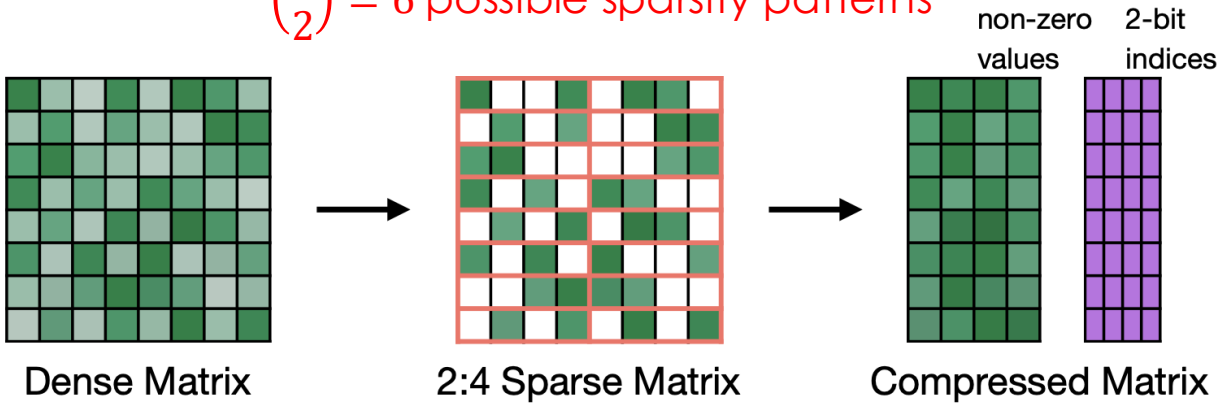
Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT

Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**
  - N:M sparsity means that in each contiguous M elements, N of them is pruned
  - A classic case is 2:4 sparsity (50% sparsity)
  - It is supported by NVIDIA's Ampere GPU Architecture, which delivers up to 2x speed up

$\binom{4}{2} = 6$ possible sparsity patterns



Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT

Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**
  - N:M sparsity means that in each contiguous M elements, N of them is pruned
  - A classic case is 2:4 sparsity (50% sparsity)
  - It is supported by NVIDIA's Ampere GPU Architecture, which delivers ~2x speed up
  - Usually maintains accuracy (tested on varieties of tasks)

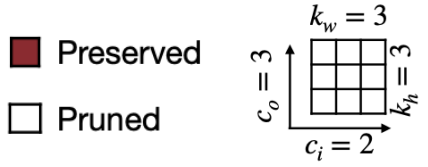
Network	Data Set	Metric	Dense FP16	Sparse FP16
ResNet-50	ImageNet	Top-1	76.1	76.2
ResNeXt-101_32x8d	ImageNet	Top-1	79.3	79.3
Xception	ImageNet	Top-1	79.2	79.2
SSD-RN50	COCO2017	bbAP	24.8	24.8
MaskRCNN-RN50	COCO2017	bbAP	37.9	37.9
FairSeq Transformer	EN-DE WMT'14	BLEU	28.2	28.5
BERT-Large	SQuAD v1.1	F1	91.9	91.9

Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT

Pruning at Different Granularities

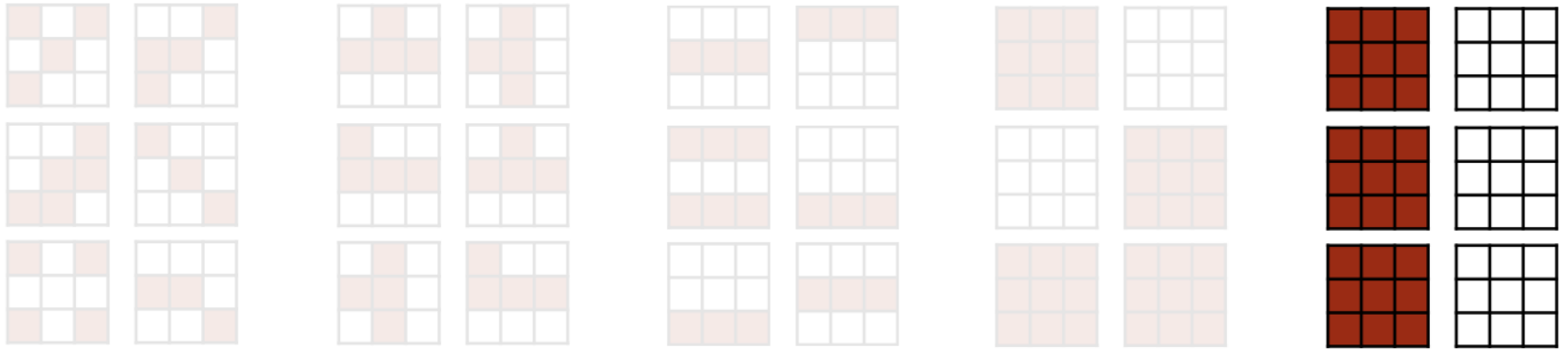
The case of convolutional layers

- Some of the commonly used pruning granularities



Pros? **Cons?**

Irregular ⟷ Regular

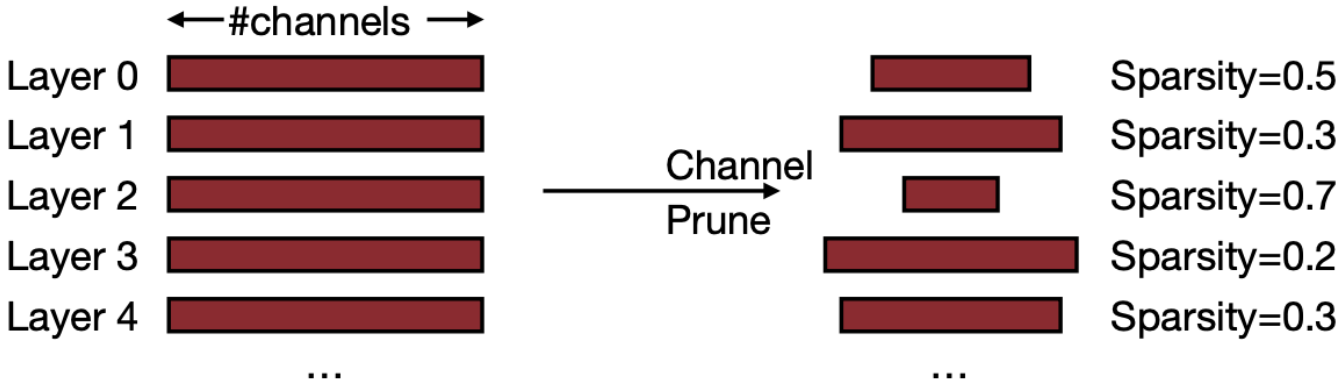


Fine-grained Pruning | Pattern-based Pruning | Vector-level Pruning | Kernel-level Pruning | Channel-level Pruning

Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**
  - Pro: Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
  - Con: smaller compression ratio

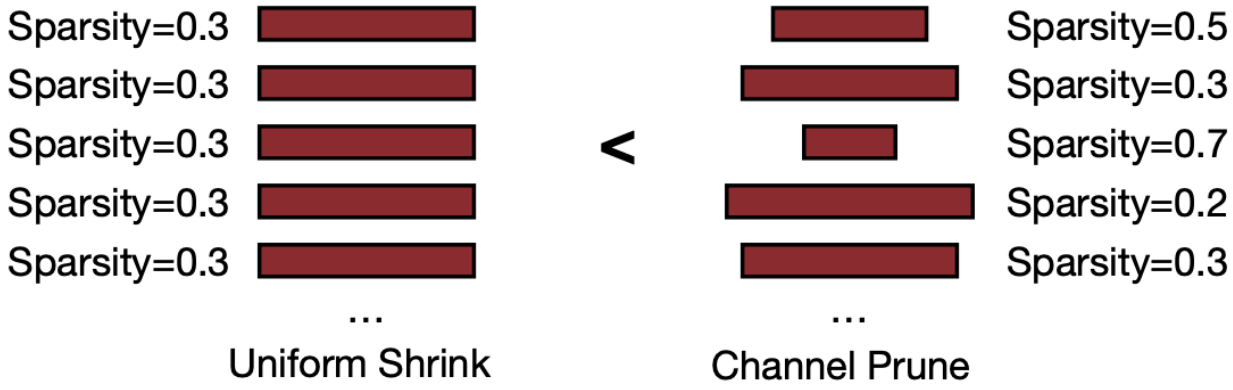


Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**
  - Pro: Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
  - Con: smaller compression ratio

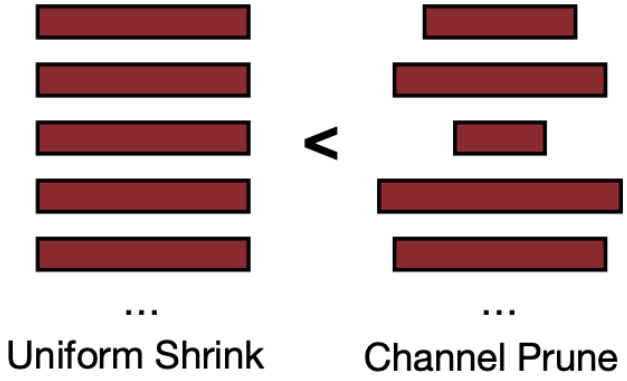
We will later discuss how to find sparsity ratios

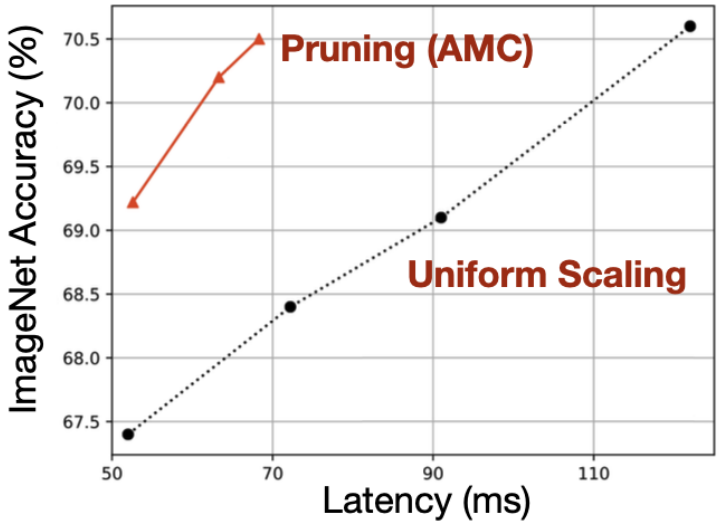


Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**
  - Pro: Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
  - Con: smaller compression ratio

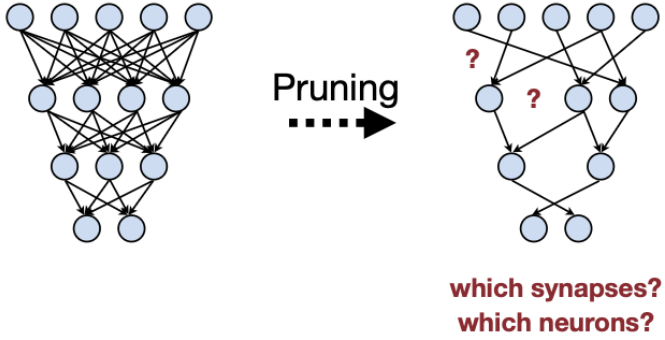




AMC: Automl for Model Compression and Acceleration on Mobile Devices [He *et al.*, ECCV 2018]

Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
  - How should we improve performance of pruned models?



Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

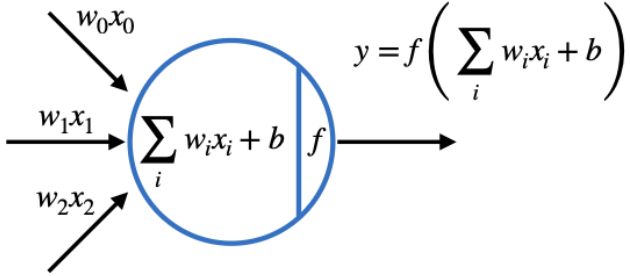
Pruning Criterion

What synapses and neurons should we prune?



Selection of Synapses to Prune

- When removing parameters from a neural network model,
  - ***the less important*** the parameters being removed are,
  - the better the performance of pruned neural network is.



Example

$$f(\cdot) = \text{ReLU}(\cdot), \quad W = [10, -8, 0.1]$$

$$\Rightarrow y = \text{ReLU}(10x_0 - 8x_1 + 0.1x_2)$$

- If one weight will be removed, which one?

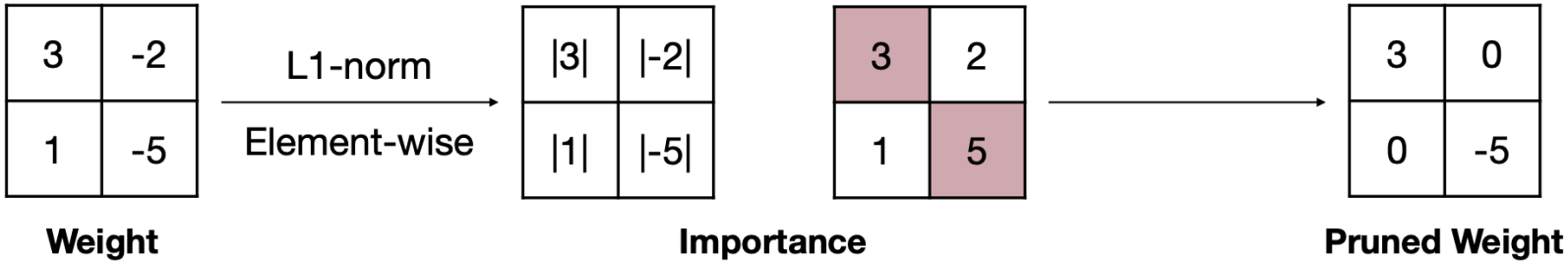
Method 1: Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
  - For element-wise pruning,

$$Importance = |W|$$

- **Example**



Learning Both Weights and Connections for Efficient Neural Network [Han *et al*., NeurIPS 2015]

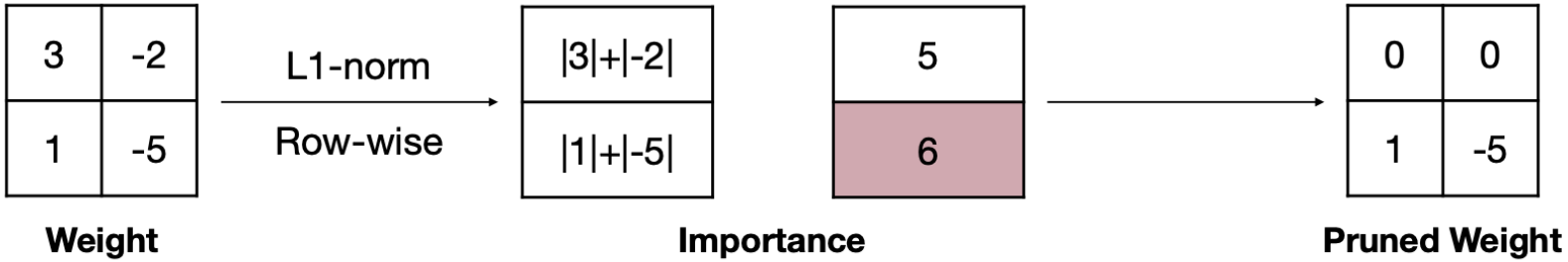
Method 1: Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
  - For row-wise pruning, the L1-norm magnitude can be defined as,

$$Importance = \sum_{i \in S} |w_i|, \text{ where } \mathbf{W}^{(S)} \text{ is the structural set } S \text{ of parameters } \mathbf{W}$$

- **Example**



Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

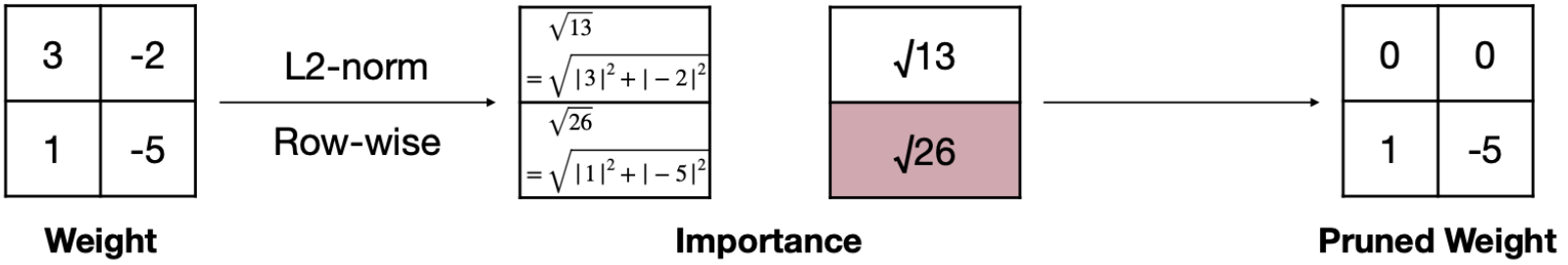
Method 1: Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
  - For row-wise pruning, the L2-norm magnitude can be defined as,

$$Importance = \sqrt{\sum_{i \in S} |w_i|^2}, \text{ where } \mathbf{W}^{(S)} \text{ is the structural set } S \text{ of parameters } \mathbf{W}$$

- **Example**



Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

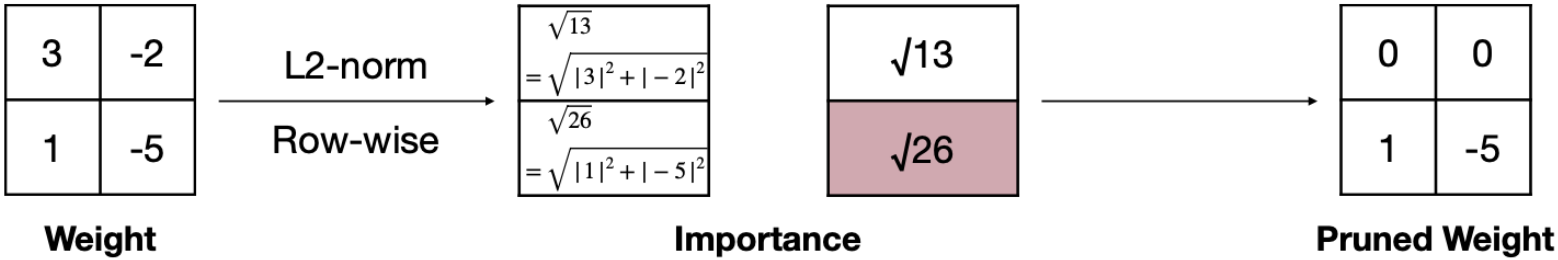
Method 1: Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
- Magnitude is also known as L_p-norm defined as,

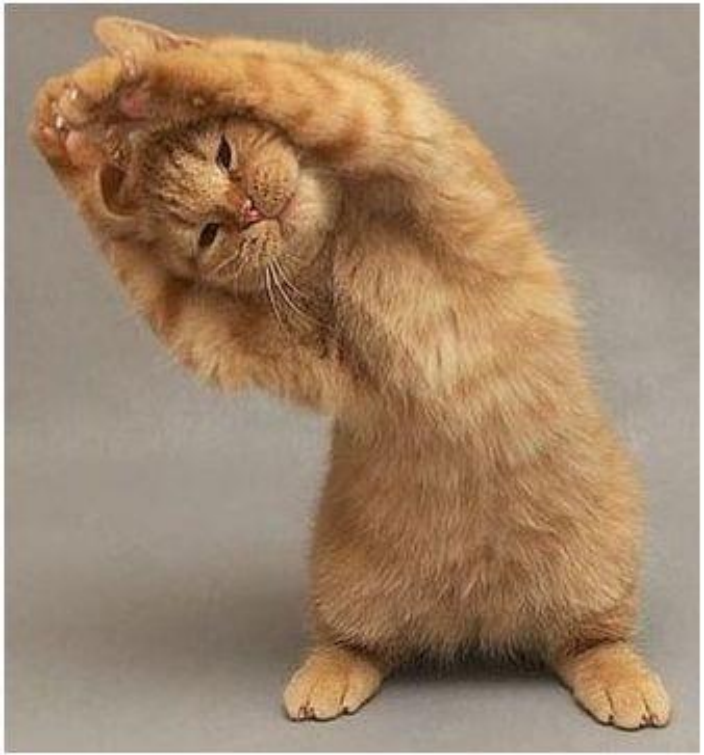
$$\|\mathbf{W}^{(S)}\|_p = \left(\sum_{i \in S} |w_i|^p \right)^{\frac{1}{p}}, \text{ where } \mathbf{W}^{(S)} \text{ is a structural set of parameters}$$

- **Example**



Learning Structured Sparsity in Deep Neural Networks [Wen *et al.*, NeurIPS 2016]

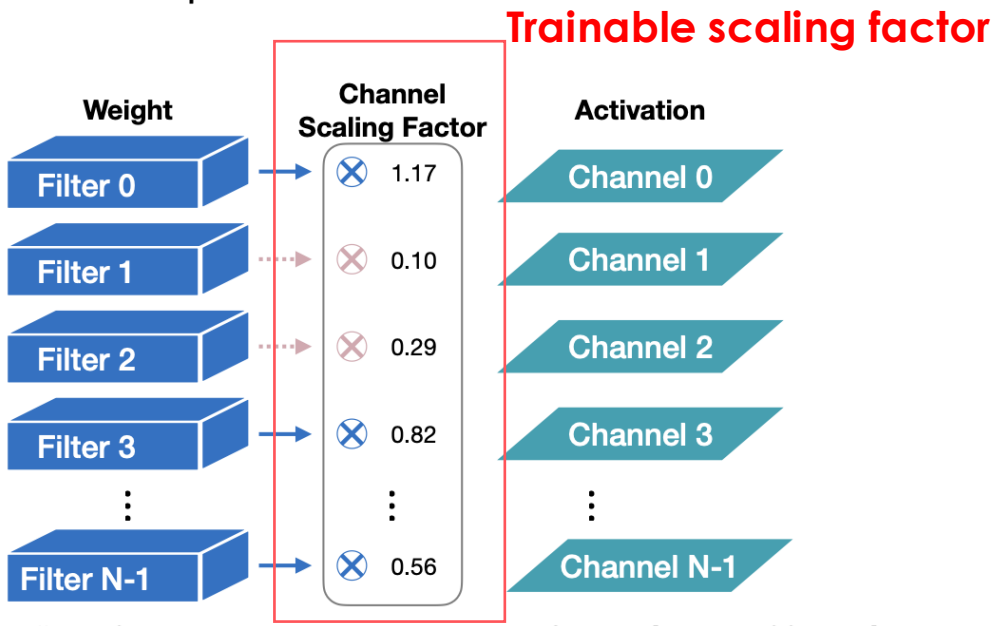
BREAK



Method 2: Scaling-based Pruning

Pruning criterion for filter pruning

- A scaling factor is associated with each filter (*i.e.*, output channel) in convolutional layers
  - The scaling factor is multiplied to the output of that channel
  - The scaling factors are trainable parameters

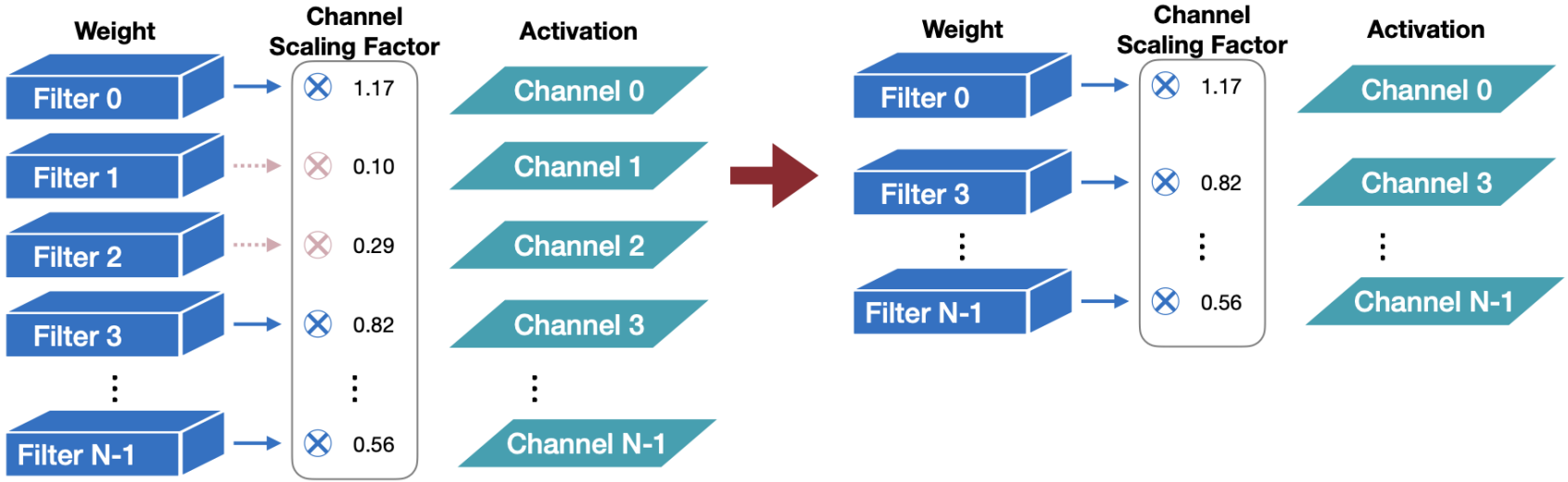


Learning Efficient Convolutional Networks through Network Slimming [Liu *et al.*, ICCV 2017]

Method 2: Scaling-based Pruning

Pruning criterion for filter pruning

- A scaling factor is associated with each filter (*i.e.*, output channel) in convolutional layers
  - The scaling factor is multiplied to the output of that channel
  - The scaling factors are trainable parameters
- The filters/output channels with small scaling factor magnitude will be pruned



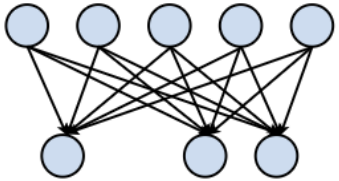
Learning Efficient Convolutional Networks through Network Slimming [Liu *et al.*, ICCV 2017]

Selection of Neurons (Activations) to Prune

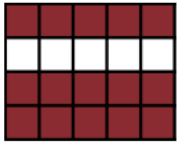
- When removing neurons from a neural network model,
  - ***the less useful*** the neurons being removed are,
  - the better the performance of pruned neural network is.

Neuron pruning is coarse-grained weight pruning

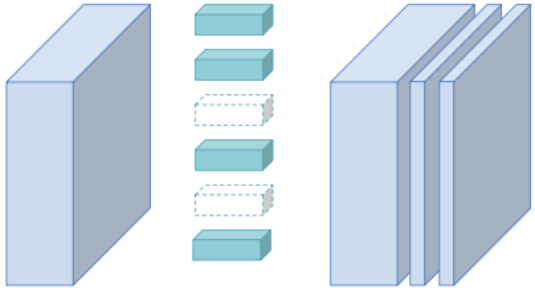
Neuron Pruning in Linear Layer

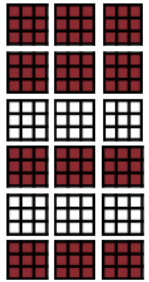


Weight Matrix

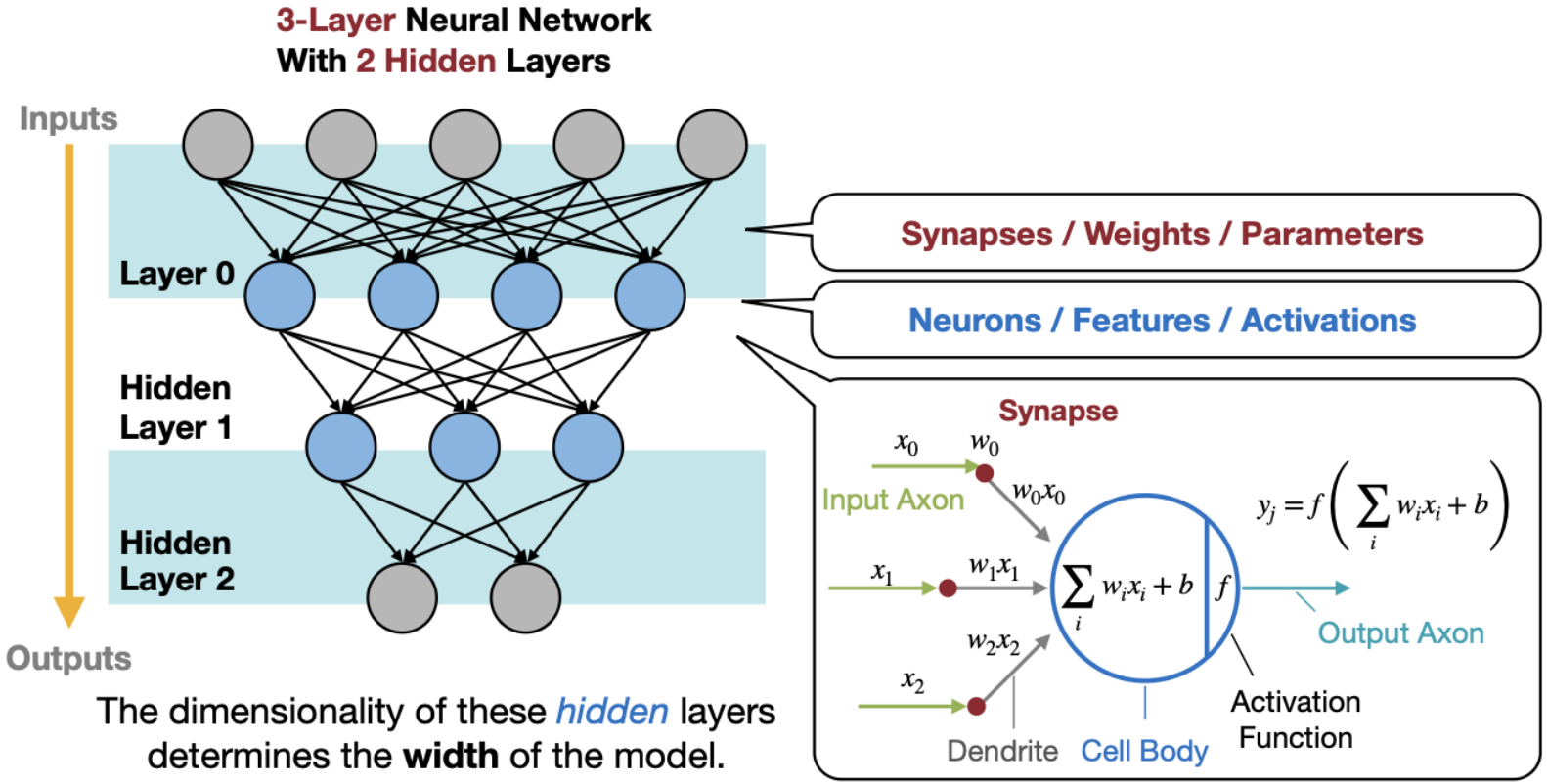


Channel Pruning in Convolution Layer





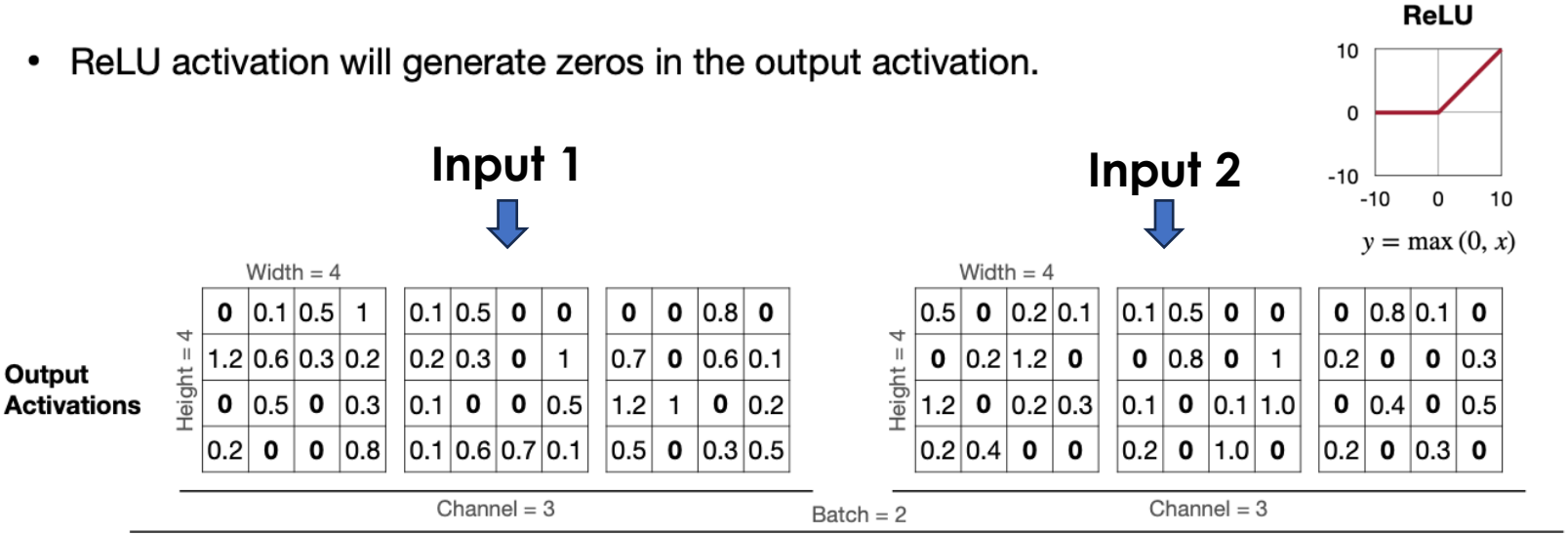
Deep Neural Network



The dimensionality of these *hidden* layers determines the **width** of the model.

Percentage-of-Zero-Based Pruning

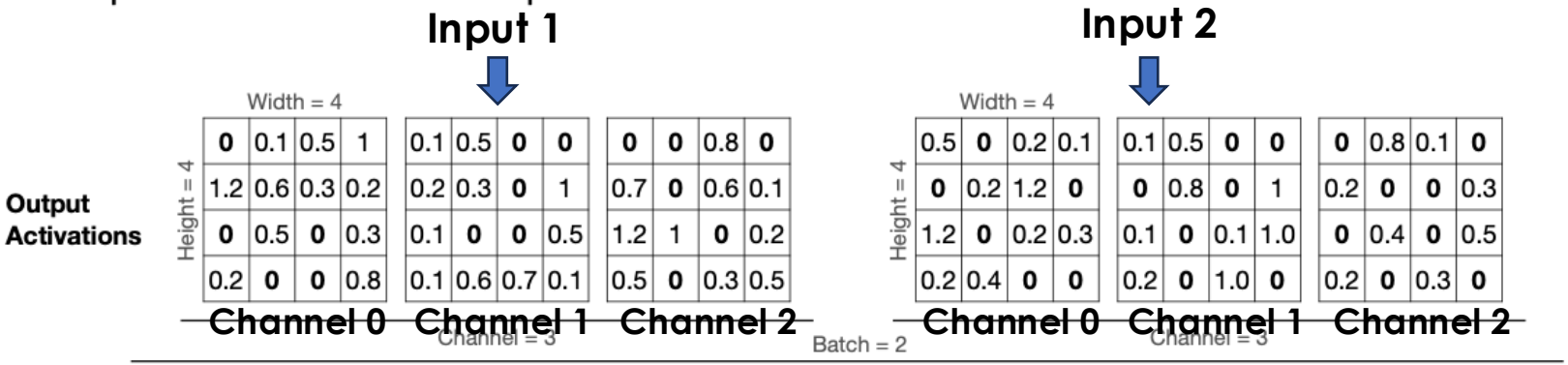
- ReLU activation will generate zeros in the output activation.



Network Trimming: A Data-Driven Neuron Pruning Approach towards Efficient Deep Architectures [Hu *et al.*, ArXiv 2017]

Percentage-of-Zero-Based Pruning

- ReLU activation will generate zeros in the output activation.
- Similar to magnitude of weights, the Average Percentage of Zero activations (APoZ) can be exploited to measure the importance of the neurons.



Average Percentage of Zeros (APoZ)

$$= \frac{5+6}{2 \cdot 4 \cdot 4} = \frac{11}{32}$$ Channel 0

$$= \frac{5+7}{2 \cdot 4 \cdot 4} = \frac{12}{32}$$ Channel 1

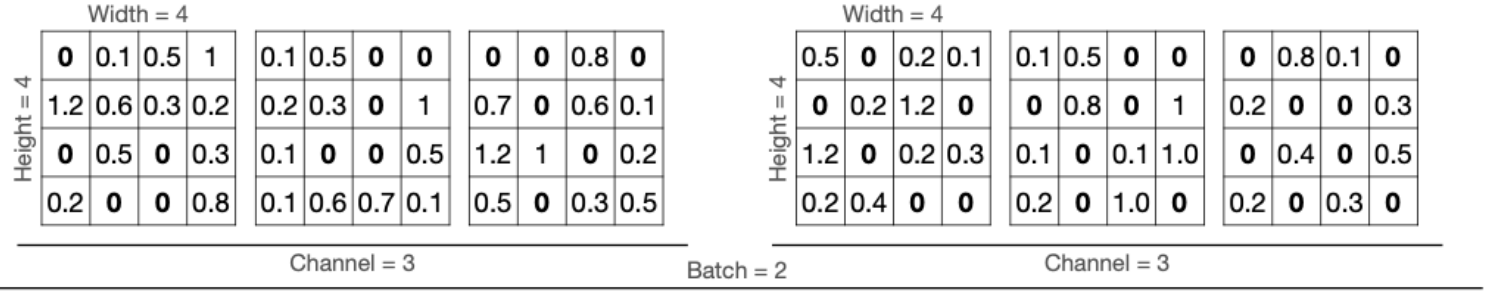
$$= \frac{6+8}{2 \cdot 4 \cdot 4} = \frac{14}{32}$$ Channel 2

Network Trimming: A Data-Driven Neuron Pruning Approach towards Efficient Deep Architectures [Hu *et al.*, ArXiv 2017]

Percentage-of-Zero-Based Pruning

- ReLU activation will generate zeros in the output activation.
- Similar to magnitude of weights, the Average Percentage of Zero activations (APoZ) can be exploited to measure the importance of the neurons.
- The smaller APoZ is, the more importance the neuron has.

Output Activations



Average Percentage of Zeros (APoZ)

$= \frac{5+6}{2 \cdot 4 \cdot 4} = \frac{11}{32}$ Channel 0

$= \frac{5+7}{2 \cdot 4 \cdot 4} = \frac{12}{32}$ Channel 1

$= \frac{6+8}{2 \cdot 4 \cdot 4} = \frac{14}{32}$ Channel 2

You must really run a few inferences so that you can calculate the average percentage of zeros

Network Trimming: A Data-Driven Neuron Pruning Approach towards Efficient Deep Architectures [Hu *et al.*, ArXiv 2017]

Labs

- 9/14: Lab1 – LM-Meter Tutorial
- 9/16: Lab 2 – LLM deployment on smartphone/laptop
- 9/21: Lab3: On-device LLM profiling



Huirong Chai
Ph.D. student @ AMAI Lab

PyTorch

MACHINE LEARNING COMPILATION

TensorFlow

At least a Linux/MacOS Laptop is required

- Processor: x86 (Intel/AMD) or ARM (Apple M1/M2)
- Memory: 8 GB

Project (Very Important !!!)

- **Up to 3 students/team**
- Your proposed project **must** fall under one of the provided topics. It can be an extension of a listed topic, a focused subset of it, or a closely related idea that aligns with the overall theme of the course.
- Deliverables:
  - Project proposal **(Due in class on 9/14)**
  - Midterm Progress Report: Preliminary results and challenges encountered
  - Presentation/Demo: Showcase final results with visualizations showing efficiency bottlenecks and trade-offs **(11/17, 11/19)**
  - Final Report (**at least 10 pages**): Methodology, results, analysis, and reflection on efficiency vs. accuracy trade-offs
  - Code: Well-documented implementation shared via GitHub



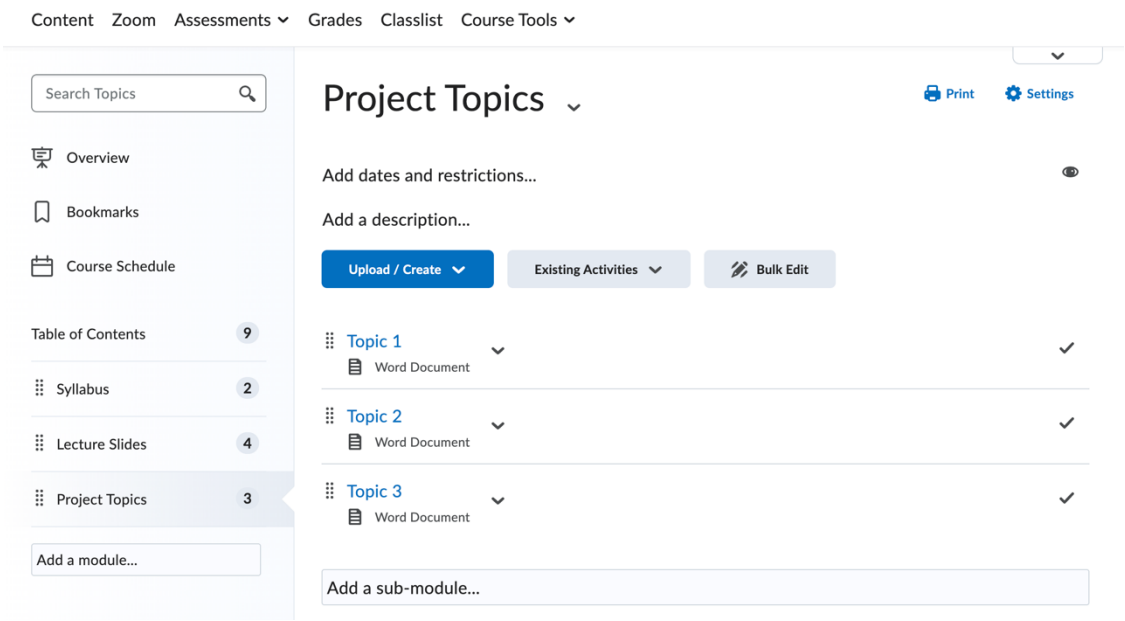
Project (Very Important !!!)

- Topic 1: Profiling LLMs on Edge Devices
- Topic 2: Studying Prompt Keyword Effects on LLM Decode Efficiency
- Topic 3: Comparing Quantization Across Hardware

Note: Choose one topic, enter your information in the Google Sheet (scan the QR code), and submit your proposal in class on 9/14.

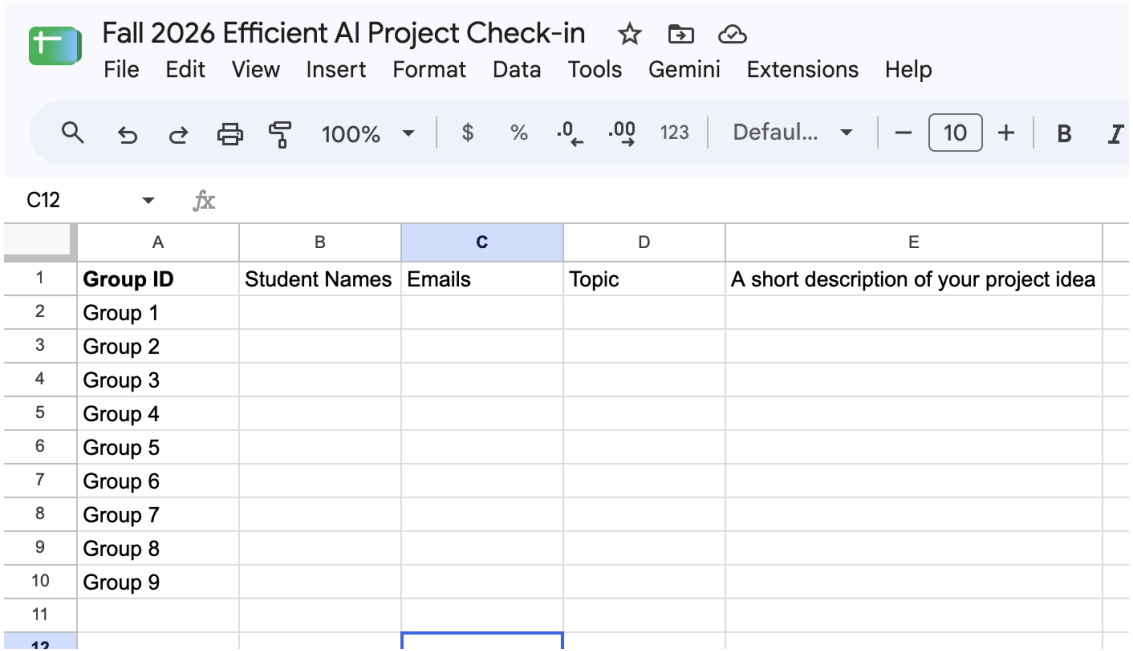


Project (Very Important !!!)



Note: Choose one topic, enter your information in the Google Sheet (scan the QR code), and submit your proposal in class on 9/14.

Project (Very Important !!!)



	A	B	C	D	E
1	**Group ID**	Student Names	Emails	Topic	A short description of your project idea
2	Group 1				
3	Group 2				
4	Group 3				
5	Group 4				
6	Group 5				
7	Group 6				
8	Group 7				
9	Group 8				
10	Group 9				
11					

Note: Choose one topic, enter your information in the Google Sheet (scan the QR code), and submit your proposal in class on 9/14.